

About this Continuous Assessment

Assessment	9
Delivery method	Canvas submission of this file after completion
Deadline	As specified on Canvas.
ILOs	Be able to implement a PD-based controller and an inverse dynamics controller for manipulators to track a desired trajectory in joint space.

Allowed packages

Do not modify these. Do not add or remove packages.

```
In [ ]: # Helper functions and packages
%pip install numpy
%pip install matplotlib
```

Allowed imports

Do not modify, add, or remove imports. Do not add imports anywhere else in this notebook.

```
In [ ]: # Helper functions and packages
import numpy as np
import matplotlib.pyplot as plt
```

Instructions

This assessment is made of 2 tasks. There are two points in total for this part of the assessment.

Each cell is tagged for the assessment, so do not delete or otherwise tamper with the cells. The assignment is to add content to each of the Python cells as requested.

Verify the spreadsheet

Your student ID will be used to assign you parameters. For each task, look into your row to see what you must implement.

Dynamics of a 2-DoF Planar Manipulator

Consider the planar manipulator equipped with two revolute joints as depicted.

- The joint angles are denoted by q_i , where $i = 1, 2$, and serve as the generalized coordinates.
- The mass of link i is represented by m_i , while l_i indicates the length of link i .
- The distance from the preceding joint to the center of mass of link i is denoted by l_{ci} , and
- The moment of inertia of link i about an axis perpendicular to the page is I_i , passing through the center of mass of link i .

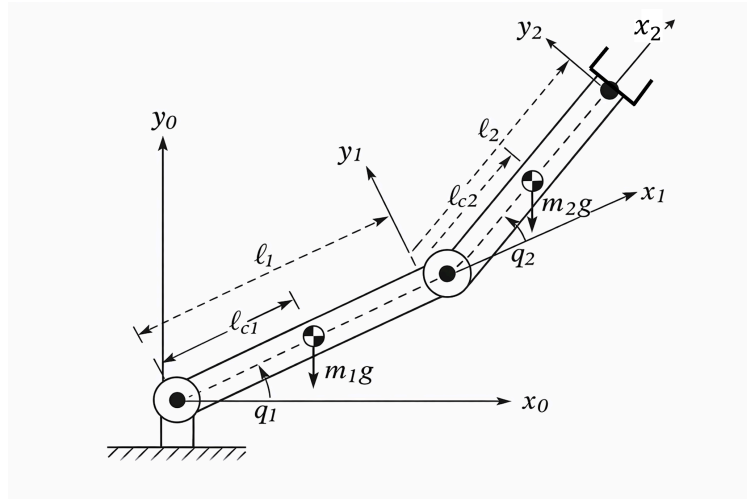


Figure 1 - A 2-DoF Planar Manipulator.

The equations of motion of the manipulator are given by:

$$D(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau,$$

where $D(q)$ is the inertia matrix, $C(q, \dot{q})\dot{q}$ represents the Coriolis and centrifugal effects, $G(q)$ is the gravity torque vector, and τ is the vector of joint control torques.

Please input the model parameters corresponding to your assigned ID, as provided in the spreadsheet.

Then, implement the required controllers by specifying the torque inputs for Tasks 1 and 2.

```
In [ ]: # Define Parameters for Tasks 1 and 2
#####
# TODO:
# Enter the parameters below (Find them in the spreadsheet)
model_parameters = {
    'm1': None, # mass of link 1
    'm2': None, # mass of link 2
    'l1': None, # length of link 1
    'l2': None, # length of link 2
    'lc1': None, # distance from preceding joint to the centre of mass of Link 1
    'lc2': None, # distance from preceding joint to the centre of mass of Link 2
    'I1': None, # inertia of link 1, kg m^2
```

```

    'I2': None, # inertia of link 2, kg m^2
    'g': None,  # gravity acceleration
}
#####

```

```

In [ ]: # Define initial joint angles
#####
# TODO:
# Enter the initial joint angles below (Find them in the spreadsheet)
# The angles are given in degrees, convert them into radians
q1_0 = None          # initial angle of joint 1
q2_0 = None          # initial angle of joint 2
#####
q_start = np.array([q1_0, q2_0])
q1_dot_0 = 0          # initial angular velocity of joint 1
q2_dot_0 = 0          # initial angular velocity of joint 2

```

```

In [ ]: # Function Library: Dynamics models
# You have worked on these functions from previous tutorials.
# You don't need to modify any of these functions here.

def Inertial_Matrix(q1, q2, params):
    m1 = params.get('m1')
    m2 = params.get('m2')
    I1 = params.get('I1')
    I2 = params.get('I2')
    l1 = params.get('l1')
    lc1 = params.get('lc1')
    lc2 = params.get('lc2')
    c2 = np.cos(q2)
    d22 = I2 + m2 * lc2**2
    d11 = d22 + I1 + m1 * lc1**2 + m2 * (l1**2 + 2 * l1 * lc2 * c2)
    d12 = d22 + m2 * l1 * lc2 * c2
    D = np.array([[d11, d12], [d12, d22]])
    return D

def coriolis(q1,q2, q1_dot, q2_dot, params):
    m2 = params.get('m2')
    l1 = params.get('l1')
    lc2 = params.get('lc2')
    s2 = np.sin(q2)
    C = np.zeros(2) # Creates a 1D array with 2 elements, initialized to 0
    h = -m2 * l1 * lc2 * s2
    p1 = h
    p2 = -h
    C[0] = p1 * (q2_dot**2 + 2 * q1_dot * q2_dot)
    C[1] = p2 * q1_dot**2
    return C

def gravity(q1,q2,params):
    m1 = params.get('m1')
    m2 = params.get('m2')
    l1 = params.get('l1')
    lc1 = params.get('lc1')
    lc2 = params.get('lc2')
    g = params.get('g')

    c1 = np.cos(q1)
    c12 = np.cos(q1+q2)

```

```

G = np.zeros(2) # Creates a 1D array with 2 elements, initialized to 0
G[0] = (m1*lc1+ m2*l1)*g*c1 + m2*lc2*g*c12
G[1] = m2*lc2*g*c12
return G

# Numerical integration
def UpdateAngle(q, qdot, dt):
    return np.array([q[0] + dt * qdot[0],
                    q[1] + dt * qdot[1]])

def UpdateVel(qdot, qddot, dt):
    return np.array([qdot[0] + dt * qddot[0],
                    qdot[1] + dt * qddot[1]])

def JointAccel(Torque, D, Cqdot, G):
    return np.linalg.inv(D) @ (Torque - Cqdot-G)

```



Task 1 - PD control with Gravity Compensation

Develop a Proportional-Derivative (PD) controller with gravity compensation to regulate the manipulator to the desired angles $q_d = [0, 0]^T$ degrees. Simulate the dynamic response of the controlled manipulator. (1 pt)

```

In [ ]: # Code for Task 1
# Define the linear PD controller with gravity compensation and simulate the sys

# Initialisation
# Define sampling rate
dt = 0.01 # DEFINE HERE THE SAMPLING RATE

# Define movement duration
T = 10 # DEFINE HERE THE MOVEMENT DURATION

# Calculate the number of samples
T_samples = int(T / dt)

#####
# TODO:
# Define desired joint angles
qd = None # Desired joint angles
#####

# Initialize q and qdot arrays
q = np.zeros((T_samples, 2))
q[0, :] = q_start # Set the first row of q to q_start

qdot = np.zeros((T_samples, 2))

# Initialize torque
Torque = np.array([0, 0])
# Initialize joint accelerations
qddot = np.array([0, 0])

# PD constants: Proportional and Derivative
Kp = 10 # Same for both joints

```

```

Kd = 2 # Same for both joints

for i in range(T_samples-1):

    # Compute dynamics
    D = Inertial_Matrix(q[i, 0], q[i, 1], model_parameters)
    Cqdot = coriolis(q[i, 0], q[i, 1], qdot[i, 0], qdot[i, 1], model_parameters)
    G = gravity(q[i,0], q[i,1],model_parameters)
    #####
    # TODO:
    # Define torque control command
    Torque = None
    #####

    qddot = JointAccel(Torque, D, Cqdot, G)

    # Movement integration
    qdot[i + 1, :] = UpdateVel(qdot[i, :], qddot, dt)
    q[i + 1, :] = UpdateAngle(q[i, :], qdot[i + 1, :], dt)

# Marking variables (don't modify)
ansT1_q = q

# End of code for Task 1

```

```

In [ ]: plt.figure()
plt.subplot(211)
plt.plot(np.linspace(0,T,len(q[:,0])),q[:,0]*180/np.pi)
plt.plot(np.linspace(0,T,len(q[:,0])),np.linspace(qd[0],qd[0],len(q[:,0])),lines
plt.legend(['Real', 'Desired'])
plt.ylabel('q1')
plt.subplot(212)
plt.plot(np.linspace(0,T,len(q[:,0])),q[:,1]*180/np.pi)
plt.plot(np.linspace(0,T,len(q[:,0])),np.linspace(0,0,len(q[:,0])),linestyle='--'
plt.ylabel('q2')
plt.show()

```



Task 2 - Inverse Dynamics Control

Implement an inverse dynamics controller to guide the manipulator to track a sinusoidal joint-space trajectory.

The desired trajectory is defined as:

$$q_d(t) = q_0 + A \sin(\omega t),$$

where q_0 is the initial joint configuration, A is the amplitude, and ω is the angular frequency.

Design and implement the joint-space inverse dynamics controller to track this trajectory, and simulate the dynamic response of the controlled manipulator. (1 pt)

```

In [ ]: # Codes for Task 2
        # Define an inverse dynamics controller and simulate the system response

```

```

# Initialisation
# Sampling time
dt = 0.01

# Movement duration
T = 10

# Number of simulation samples
T_samples = int(T / dt)

# Time vector
t = np.linspace(0, T, T_samples)

# Initialise joint position and velocity arrays
q = np.zeros((T_samples, 2))
q[0, :] = q_start          # Initial joint configuration

qdot = np.zeros((T_samples, 2))

# Initialize torque
Torque = np.array([0, 0])

# Initialize joint accelerations
qddot = np.array([0, 0])

#####
# Desired sinusoidal trajectory
# The desired trajectory is:
#  $q_d(t) = q_{\text{offset}} + A * \sin(w * t)$ 
# TODO: define
# qd      : desired joint position
# qd_dot  : desired joint velocity
# qd_ddot : desired joint acceleration

A = np.array([0.5, 0.5])      # Amplitude of each joint trajectory, in rad
w = np.array([1.0, 1.0])     # Angular frequency of each joint trajectory,
q_offset = q_start           # Sinusoidal trajectory is centred at q_start

qd = np.zeros((T_samples, 2))
qd_dot = np.zeros((T_samples, 2))
qd_ddot = np.zeros((T_samples, 2))

for i in range(T_samples):
    qd[i, :] = None
    qd_dot[i, :] = None
    qd_ddot[i, :] = None
#####

# Controller gains

Kp = 10 # Same for both joints
Kd = 2  # Same for both joints

# Simulation loop

for i in range(T_samples - 1):
    # Compute robot dynamics
    D = Inertial_Matrix(q[i, 0], q[i, 1], model_parameters)
    Cqdot = coriolis(

```

```

        q[i, 0], q[i, 1],
        qdot[i, 0], qdot[i, 1],
        model_parameters
    )
    G = gravity(q[i, 0], q[i, 1], model_parameters)

#####
# TODO: Implement inverse dynamics controller for joint-space trajectory tra
# Inverse dynamics control law
Torque = None
#####

# Compute joint acceleration
qddot = JointAccel(Torque, D, Cqdot, G)

# Numerical integration
qdot[i + 1, :] = UpdateVel(qdot[i, :], qddot, dt)
q[i + 1, :] = UpdateAngle(q[i, :], qdot[i + 1, :], dt)

# Marking variables: do not modify
ansT2_q = q

# End of codes for Task 2

```

```

In [ ]: plt.figure()
plt.subplot(211)
plt.plot(t,np.rad2deg(q[:,0]))
plt.plot(t,np.rad2deg(qd[:,0]),linestyle='--')

plt.legend(['Real','Desired'])
plt.ylabel('q1')
plt.subplot(212)
plt.plot(t,np.rad2deg(q[:,1]))
plt.plot(t,np.rad2deg(qd[:,1]),linestyle='--')
plt.ylabel('q2')
plt.show()

```